

Backend REST API con Node.js & Express

Sistema di Gestione Menu Ristorante

Informazioni esercizio	
Livello	Intermedio / Avanzato
Prerequisiti	JavaScript ES6+, concetti REST, SQL, async/await
Tecnologie	Node.js 20+, Express 4, Sequelize 6, MariaDB
Endpoint richiesti	15 endpoint totali
Consegna	Codice sorgente, script SQL (creazione tabelle + dati), collection Postman

1. Obiettivi dell'esercizio

Lo studente dovrà progettare e implementare un backend REST per la gestione del menu di un ristorante, usando Node.js con Express e Sequelize come ORM. Il progetto copre la modellazione relazionale con due relazioni di tipo diverso (multi-a-uno e multi-a-molti), la gestione delle operazioni CRUD complete e la scrittura di query di interrogazione più mirate.

Al termine dell'esercizio lo studente sarà in grado di:

- Definire modelli Sequelize con relazioni belongsTo/hasMany (multi-a-uno) e belongsToMany (multi-a-molti)
- Implementare tutti gli endpoint CRUD per più risorse collegate tra loro
- Gestire correttamente le operazioni asincrone con async/await
- Scrivere query di interrogazione avanzate tramite Sequelize (ricerca, filtri, relazioni annidate)
- Produrre script SQL per la creazione delle tabelle e il popolamento dei dati di test
- Organizzare e documentare le chiamate API in una collection Postman

2. Struttura del database

Il database MariaDB deve contenere tre tabelle. La tabella principale è Piatto. Essa è collegata a Categoria con una relazione multi-a-uno (ogni piatto appartiene a una sola categoria, ma una categoria può contenere molti piatti) e a Ingrediente con una relazione multi-a-molti (un piatto è composto da più ingredienti, uno stesso ingrediente può comparire in molti piatti).

2.1 Tabella principale — Piatto

Rappresenta un piatto presente nel menu del ristorante.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	NOT NULL	Nome del piatto, es. "Carbonara", "Tiramisù"
descrizione	TEXT	opzionale	Descrizione del piatto e degli ingredienti principali
prezzo	FLOAT	NOT NULL	Prezzo del piatto in euro

Campo	Tipo	Vincoli	Note
vegetariano	BOOLEAN	default false	Indica se il piatto è vegetariano
disponibile	BOOLEAN	default true	Indica se il piatto è nel menu attivo
CategoriaId	INTEGER	FK → Categoria	Relazione molti-a-uno (Sequelize: belongsTo)

2.2 Tabella secondaria — Categoria (relazione Multi-a-Uno con Piatto)

Rappresenta una categoria del menu (es. Antipasti, Primi, Secondi, Dolci). Una categoria può contenere molti piatti; ogni piatto appartiene a una sola categoria.

In Sequelize: `Categoria.hasMany(Piatto)` e `Piatto.belongsTo(Categoria)`.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	UNIQUE, NOT NULL	Es. "Antipasti", "Primi", "Secondi", "Dolci"
descrizione	TEXT	opzionale	Descrizione estesa della categoria
ordineMenu	INTEGER	NOT NULL	Ordine di visualizzazione della categoria nel menu

2.3 Tabella secondaria — Ingrediente (relazione Multi-a-Molti con Piatto)

Rappresenta un ingrediente utilizzato nei piatti. Un piatto è composto da più ingredienti; uno stesso ingrediente può comparire in molti piatti. La relazione è gestita da una tabella associativa `PiattoIngrediente` (`PiattoId`, `IngredienteId`).

In Sequelize: `Piatto.belongsToMany(Ingrediente, { through: 'PiattoIngrediente' })` e `Ingrediente.belongsToMany(Piatto, { through: 'PiattoIngrediente' })`.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
nome	STRING	UNIQUE, NOT NULL	Es. "Pomodoro", "Mozzarella", "Guanciale"
allergene	BOOLEAN	default false	Indica se l'ingrediente è un allergene comune
unitaMisura	STRING	opzionale	Unità di misura, es. "g", "ml", "pz"

Nota sul diagramma ER

Prima di scrivere codice, disegnare il diagramma ER delle tre tabelle e identificare esplicitamente: chiavi primarie, chiavi esterne, cardinalità di ogni relazione e i metodi Sequelize corrispondenti (`belongsTo/hasMany` per la relazione con `Categoria`, `belongsToMany` per la relazione con `Ingrediente`).

3. Tecnologie e struttura del progetto

3.1 Stack tecnologico richiesto

- Node.js 20 LTS o superiore
- Express 4 — framework web per la creazione di REST API
- Sequelize 6 — ORM per Node.js compatibile con MariaDB, MySQL, PostgreSQL
- mariadb — driver nativo Node.js per MariaDB (alternativa: mysql2)
- dotenv — gestione delle variabili di ambiente per le credenziali del database

3.2 Struttura consigliata del progetto

```

ristorante/
├── app.js                # entry point Express
├── config/
│   └── database.js      # connessione Sequelize + MariaDB
├── models/
│   ├── index.js        # carica modelli e associazioni
│   ├── Piatto.js       # modello Sequelize Piatto
│   ├── Categoria.js    # modello Sequelize Categoria
│   └── Ingrediente.js  # modello Sequelize Ingrediente
├── routes/
│   ├── piatti.js       # router endpoint piatti
│   ├── categorie.js    # router endpoint categorie
│   └── ingredienti.js  # router endpoint ingredienti
├── sql/
│   ├── create_tables.sql # script di creazione tabelle
│   └── seed_data.sql    # script di popolamento dati
├── postman/
│   └── ristorante.json  # collection Postman
├── .env                # credenziali DB (non committare!)
├── package.json
└── README.md

```

4. Elenco degli endpoint richiesti

Il progetto deve esporre esattamente 15 endpoint. Ogni endpoint deve restituire codici HTTP appropriati (200, 201, 204, 400, 404) e rispondere sempre con JSON.

4.1 Endpoint — Categorie (3 endpoint)

#	Metodo	URL	Descrizione	Tag
1	GET	/api/categorie	Restituisce l'elenco di tutte le categorie (ordinate per ordineMenu)	Categoria
2	POST	/api/categorie	Crea una nuova categoria	Categoria
3	DELETE	/api/categorie/:id	Elimina una categoria (solo se senza piatti associati)	Categoria

4.2 Endpoint — Ingredienti (4 endpoint)

#	Metodo	URL	Descrizione	Tag
4	GET	/api/ingredienti	Restituisce l'elenco di tutti gli ingredienti	Ingrediente
5	POST	/api/ingredienti	Crea un nuovo ingrediente	Ingrediente
6	GET	/api/ingredienti/:id	Dettaglio ingrediente con i piatti che lo utilizzano	Ingrediente
7	PUT	/api/ingredienti/:id	Aggiorna i dati di un ingrediente esistente	Ingrediente

4.3 Endpoint — Piatti (5 endpoint CRUD)

#	Metodo	URL	Descrizione	Tag
8	GET	/api/piatti	Lista di tutti i piatti con categoria e ingredienti	Piatto
9	POST	/api/piatti	Crea un nuovo piatto (con array di id ingredienti)	Piatto
10	GET	/api/piatti/:id	Dettaglio completo di un singolo piatto	Piatto
11	PUT	/api/piatti/:id	Modifica i dati di un piatto esistente	Piatto
12	DELETE	/api/piatti/:id	Elimina un piatto dal menu	Piatto

4.4 Endpoint — Interrogazioni avanzate (3 endpoint)

#	Metodo	URL	Descrizione	Tag
13	GET	/api/piatti/search	Ricerca piatti per nome (query param: ?q=...)	Ricerca
14	GET	/api/piatti/vegetariani	Lista dei soli piatti vegetariani e disponibili	Ricerca
15	GET	/api/categorie/:id/piatti	Tutti i piatti di una categoria specifica	Ricerca

5. Specifiche dettagliate degli endpoint

Endpoint 9 — POST /api/piatti

Il body JSON della richiesta deve includere:

- nome, prezzo (obbligatori)
- descrizione, vegetariano, disponibile (opzionali)
- CategoriaId — FK verso la tabella Categoria (obbligatorio)
- ingredientiIds — array di interi con gli ID degli ingredienti da associare al piatto (opzionale)

Prima di creare il piatto, verificare che la categoria con CategoriaId esista, altrimenti HTTP 404 con messaggio: "Categoria non trovata.". Dopo aver creato il piatto con `Piatto.create()`, usare il metodo Sequelize `piatto.setIngredienti(ingredientiIds)` per stabilire le associazioni multi-a-molti. Se uno degli ID ingrediente non esiste, restituire HTTP 404 con messaggio descrittivo. Risposta di successo: HTTP 201.

Endpoint 13 — GET /api/piatti/search?q=...

Deve effettuare una ricerca case-insensitive sul campo nome utilizzando l'operatore `Op.like` di Sequelize. Su MariaDB la ricerca con LIKE è già case-insensitive per default con collation `utf8_general_ci`. Esempio: `/api/piatti/search?q=pasta` deve restituire tutti i piatti il cui nome contiene "pasta". Se il parametro `q` non viene fornito, restituire HTTP 400 con messaggio: "Parametro di ricerca 'q' obbligatorio.".

Attenzione all'ordine delle route!

In Express, `/api/piatti/search` e `/api/piatti/vegetariani` devono essere definite PRIMA di `/api/piatti/:id` nel file delle route, altrimenti Express interpreterà 'search' e 'vegetariani' come valori dinamici di `:id`.

Endpoint 14 — GET /api/piatti/vegetariani

Deve restituire solo i piatti che sono contemporaneamente vegetariani e disponibili, utilizzando una clausola `where` con entrambe le condizioni (`vegetariano: true` e `disponibile: true`). Includere per ciascun piatto anche la categoria di appartenenza.

Endpoint 3 — DELETE /api/categorie/:id

Prima di eliminare la categoria, verificare che non ci siano piatti associati con `Piatto.count({ where: { CategoriaId: id } })`. Se il conteggio è maggiore di zero, restituire HTTP 400 con il messaggio: "Impossibile eliminare: esistono piatti associati a questa categoria.". Risposta di successo: HTTP 204 No Content.

6. Definizione dei modelli e delle associazioni

6.1 Associazioni in models/index.js

Dopo aver importato tutti e tre i modelli, definire le associazioni nel file `models/index.js`:

```
// Relazione molti-a-uno: Piatto → Categoria
Categoria.hasMany(Piatto, { foreignKey: 'CategoriaId' });
Piatto.belongsTo(Categoria, { foreignKey: 'CategoriaId' });

// Relazione molti-a-molti: Piatto ↔ Ingrediente
Piatto.belongsToMany(Ingrediente, { through: 'PiattoIngrediente' });
Ingrediente.belongsToMany(Piatto, { through: 'PiattoIngrediente' });
```

6.2 Include nelle query

Per restituire un piatto con la categoria e gli ingredienti associati usare l'opzione `include` nelle query Sequelize:

```
const piatti = await Piatto.findAll({
  include: [
    { model: Categoria },
    { model: Ingrediente, through: { attributes: [] } }
  ]
});
```

L'opzione `through: { attributes: [] }` esclude dalla risposta JSON i campi interni della tabella associativa `PiattoIngrediente`.

7. Script SQL richiesti

Nella cartella `sql/` devono essere presenti due file SQL eseguibili direttamente su MariaDB.

7.1 create_tables.sql

Deve contenere le istruzioni `CREATE TABLE` per tutte e quattro le tabelle (`Categorie`, `Ingredienti`, `Piatti`, `PiattoIngrediente`), con i vincoli di chiave primaria, chiave esterna e i valori di default. Usare `IF NOT EXISTS` per rendere lo script rieseguibile. Le tabelle devono essere create nell'ordine corretto per rispettare i vincoli di integrità referenziale: prima `Categorie` e `Ingredienti`, poi `Piatti`, infine `PiattoIngrediente`.

7.2 seed_data.sql

Deve contenere le istruzioni `INSERT INTO` per popolare il database con dati di test realistici. Sono richiesti almeno:

- 5 categorie con ordini di visualizzazione diversi (es. Antipasti, Primi, Secondi, Contorni, Dolci)
- 15 ingredienti, di cui alcuni contrassegnati come allergeni
- 12 piatti distribuiti tra le categorie, con prezzi variati e alcuni marcati come vegetariani o come non disponibili
- associazioni nella tabella `PiattoIngrediente` tali che ogni piatto abbia almeno un ingrediente, alcuni ingredienti compaiano in più piatti e siano presenti sia piatti vegetariani sia non vegetariani (per testare l'endpoint 14)

8. Requisiti di consegna

8.1 Obbligatori

- 1 Codice sorgente Node.js — Cartella del progetto completa e funzionante, con file `package.json` contenente tutte le dipendenze e uno script "start" per avviare il server. Il codice deve essere privo di errori e rispettare la struttura delle cartelle indicata al punto 3.2.
- 2 Script SQL — I due file `create_tables.sql` e `seed_data.sql` nella cartella `sql/`. Gli script devono essere eseguibili su un server MariaDB pulito senza alcuna modifica manuale. Verificare che l'ordine delle `CREATE TABLE` rispetti le dipendenze tra le chiavi esterne.

- Collection Postman — File .json importabile in Postman con tutti e 15 gli endpoint configurati. Per gli endpoint POST e PUT devono essere inclusi body di esempio con dati realistici. La collection deve essere organizzata in tre cartelle (Categorie, Ingredienti, Piatti) che rispecchino i tag degli endpoint.

8.2 Facoltativi (per voto eccellente)

- Endpoint aggiuntivo GET /api/piatti/senza-allergeni che restituisce i soli piatti che non contengono alcun ingrediente marcato come allergene (richiede il controllo sugli ingredienti associati).
- Paginazione (?page=1&limit=10) sugli endpoint GET di tipo lista.
- Middleware di validazione dell'input con express-validator.
- Test con Jest + Supertest per almeno 5 endpoint.

9. Criteri di valutazione

Criterio	Peso	Punti max
Correttezza modelli e relazioni Sequelize	20%	2 / 10
Funzionamento degli endpoint CRUD	30%	3 / 10
Validazione input e gestione errori HTTP	20%	2 / 10
Endpoint di interrogazione avanzata	15%	1.5 / 10
Qualità del codice e struttura progetto	15%	1.5 / 10
TOTALE	100%	10 / 10

10. Domande guidate per la progettazione

Prima di iniziare a scrivere codice, rispondere per iscritto alle seguenti domande:

- Come si gestisce in Sequelize una relazione multi-a-molti? Cos'è la tabella through e come viene creata automaticamente?
- Qual è la differenza tra `Piatto.findOne()` e `Piatto.findByPk()`? Quando useresti l'uno o l'altro?
- Perché in Express è importante l'ordine di definizione delle route? Fai un esempio concreto con `/piatti/search` e `/piatti/:id`.
- Cosa succede se si tenta di eliminare una categoria con piatti associati senza gestire l'errore? Quale vincolo di integrità referenziale entra in gioco?
- Come si implementa una ricerca case-insensitive in Sequelize su MariaDB? Che ruolo ha la collation del database?
- Perché conviene usare il metodo `setIngredienti()` invece di inserire manualmente le righe nella tabella `PiattoIngrediente`?

Avvertenza

L'uso di soluzioni generate automaticamente da strumenti di intelligenza artificiale senza comprensione del codice prodotto è considerato disonestà accademica. Il docente si riserva di richiedere la spiegazione di qualsiasi parte del materiale consegnato.